Patrones estructurales

Hoja de ejercicios (Alumno)

Patrón *Adapter*

Escenario

Se dispone de una aplicación que controla un telescopio y necesita localizar distintos tipos de objetos/estructuras en el campo de visión. Históricamente, el sistema usaba un localizador antiguo (*legacy*) con una interfaz estable.

En este sistema de localización los parámetros disponibles para localizar un cuerpo con el telescopio eran las siguientes:

- El parámetro planares, mediante el méteodo `locatePlanars()`
- El parámetro granulos, mediante el método `locateGranules()`
- El parámetro semi cuantos, mediante el método `locateSemiQuanta()`

Recientemente, se ha incorporado un nuevo sistema de localización (*modern*) con una API distinta. El problema es que el resto del programa (el cliente) está escrito para trabajar con la interfaz antigua, y queremos **evitar reescribir el cliente** cada vez que cambie el sistema de localización.

En el sistema nuevo de localización los parámetros disponibles para localizar un cuerpo con el telescopio son las siguientes:

- Alphas (mediante el método `tuneAlphas()`)
- Gammas (mediante el método `tuneGammas()`)

Se conoce cómo se implementan las operaciones de la interfaz de localización en función de las nuevas operaciones. En concreto.

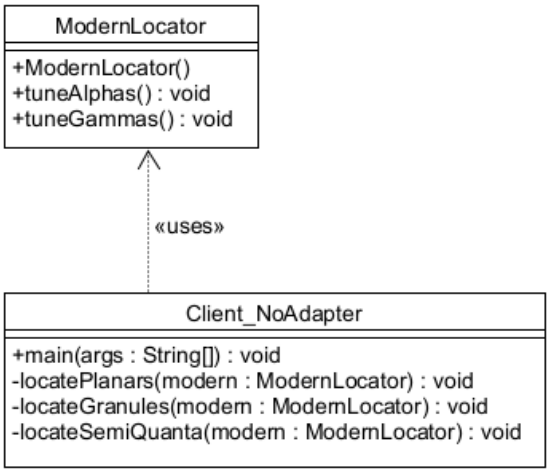
- `locatePlanars()`, se implementa con la secuencia:
  - tuneGammas()
  - tuneAlphas()
- `locateGranules()`
  - tuneAlphas()
- `locateSemiQuanta()`
  - tuneGammas()

Código proporcionado (sin patrón)

En la primera aproximación que se ha hecho para solucionar este problema, se ha seguido un enfoque que podríamos considerar "de fuerza bruta". En el fichero `Adapter_NoPattern.zip` se proporciona un proyecto Java que resuelve el escenario **sin aplicar el patrón *Adapter***. En esta versión, el propio cliente realiza la **traducción manual** entre lo que necesita (operaciones legacy) y lo que ofrece el servicio moderno. Esto introduce acoplamiento fuerte y mezcla responsabilidades.

UML de la solución sin patrón

En la versión sin patrón, el cliente termina conociendo la API del servicio moderno y aplicando conversiones a mano. El diagrama UML de esta aproximación se muestra a continuación.



Se pide

Debes crear un nuevo proyecto Java aplicando el patrón *Adapter* en concreto la opción que se basa en objetos, de modo que:

- El **cliente** trabaje únicamente con una interfaz estable (*LegacyLocator*) y no conozca la API del sistema moderno.
- La traducción entre la interfaz legacy y la API moderna quede encapsulada en un **adaptador**.
- El sistema moderno (servicio) permanezca sin cambios.

Elementos del patrón que se deben implementar (asegúrate de mantener los nombres y signaturas de los métodos)

Tu solución deberá contener, como mínimo, los siguientes elementos (nombres recomendados):

- **Interfaz esperada por el cliente:** `LegacyLocator`
  - `public void locatePlanars()`
  - `public void locateGranules()`
  - `public void locateSemiQuanta()`
- **Service (API moderna existente):** `ModernLocator`
  - `public void tuneAlphas()`
  - `public void tuneGammas()`
- **Adapter:** `ModernToLegacyLocatorAdapter`

- Debe **implementar** `LegacyLocator`.
- Debe contener internamente una referencia a `ModernLocator` (por composición).
- Debe traducir cada operación legacy (`locate...`) a una o varias llamadas al servicio moderno (`tuneAlphas`, `tuneGammas`), según corresponda.

- **Cliente (programa principal de prueba):** una clase `Client` (o equivalente)
  - Debe manejar el localizador del telescopio manteniendo la interfaz ya conocida `LegacyLocator` (no `ModernLocator`).
  - Debe crear el adaptador y ejecutar una secuencia de llamadas a `locateGranules()`, `locatePlanars()`, `locateSemiQuanta()`.

Objetivo

El objetivo principal es evidenciar la diferencia entre:

- **Sin patrón:** el cliente traduce y depende de la API moderna (acoplamiento y duplicación).
- **Con *Adapter*:** el cliente depende de una interfaz estable y la traducción queda aislada.